OAuth de raiz

Como acrescentar «Entrar com Google / Microsoft» a uma app ASP.NET Core (Blazor Server) partindo do zero: o que precisas de arranjar, o que escreves, o que a framework faz por ti, e onde é que as pessoas normalmente se magoam. Companheiro: **OAuth_Flow_AsNet.pdf** (o fluxo concreto do AsNet).

Índice

1. O modelo mental em 60 segundos
2. A lista de compras (antes de qualquer código)
3. Registar a app no Google e na Microsoft
4. Guardar os segredos como deve ser
5. A implementação recomendada (a via da framework)
6. O que a framework está a fazer por ti
7. Checklist de segurança
8. Erros clássicos e como diagnosticá-los
9. Mapa para o AsNet
10. Glossário

---

1. O modelo mental em 60 segundos

O problema: queres que a pessoa entre na tua app sem lhe pedires a password do Google — porque pedir passwords alheias é inaceitável para todos os envolvidos. A solução: a tua app *delega a verificação de identidade* ao Google e recebe de volta uma prova assinada de quem a pessoa é.

A analogia do hotel: a receção (Google) verifica o cartão de cidadão e entrega um cartão-chave; a porta do quarto (a tua app) só alguma vez vê o cartão-chave. E o ponto que resolve 90% da confusão: **o OAuth só responde a «quem é?»**. «Pode entrar?», «a que empresa pertence?», «o que pode ver?» — isso é autorização, é problema teu, e acontece *depois* do OAuth terminar.

- **OAuth 2.0** — o protocolo de delegação (o mecanismo do «voucher»).
- **OpenID Connect (OIDC)** — uma camada em cima do OAuth 2.0 que o especializa em identidade: acrescenta o `id_token` (um JWT assinado que descreve a pessoa) e o scope `openid`. Quando fazes «login social», estás quase sempre a usar OIDC.

---

2. A lista de compras (antes de qualquer código)

1. **Um registo de aplicação em cada fornecedor** — dá-te o par `ClientId` (público, identifica a app) + `ClientSecret` (privado, prova que o pedido vem mesmo do teu servidor).
2. **Os redirect URIs exatos** — o(s) endereço(s) para onde o fornecedor pode devolver o browser. Têm de estar registados *caráter a caráter* (esquema, host, porta, path): `https://localhost:5001/signin-google` para dev e `https://app.cliente.com/signin-google` para produção são DOIS registos.

3. **Decidir os scopes** — para login chegam `openid email profile`. Não peças mais do que precisas: cada scope extra é um pedido de consentimento e uma responsabilidade.
4. **Pacotes NuGet**:

   ```
   dotnet add package Microsoft.AspNetCore.Authentication.Google
   dotnet add package Microsoft.AspNetCore.Authentication.OpenIdConnect
   ```

   O da Google traz o handler específico; o OpenIdConnect genérico serve a Microsoft (Entra ID) e qualquer outro fornecedor OIDC.
5. **Um sítio seguro para os segredos** — secção 4.

---

3. Registar a app no Google e na Microsoft

Google (console.cloud.google.com)

1. Cria (ou escolhe) um projeto → **APIs & Services** → **OAuth consent screen**: tipo *External*, nome da app, email de suporte. Para login básico não precisas de verificação da Google.
2. **Credentials** → **Create credentials** → **OAuth client ID** → tipo *Web application*.
3. Em **Authorized redirect URIs** adiciona os teus callbacks (dev e prod). O default do handler ASP.NET é `/signin-google`.
4. Guarda o `Client ID` e o `Client secret` que a consola te mostra.

Microsoft (portal.azure.com → Microsoft Entra ID)

1. **App registrations** → **New registration**. «Supported account types» é a decisão importante:
   - *Accounts in any organizational directory* = `TenantId: organizations` — qualquer conta de trabalho (é o que o AsNet usa);
   - + contas pessoais = `common`; só o teu tenant = o GUID do tenant.
2. Em **Redirect URIs** (plataforma *Web*) adiciona os callbacks (no AsNet: `/auth/mobile-login-callback`).
3. **Certificates & secrets** → **New client secret** — copia o *Value* na hora (nunca mais volta a aparecer) e aponta a validade: os secrets da Microsoft **expiram** (6–24 meses) e o login morre silenciosamente no dia em que expiram. Põe um lembrete.

---

4. Guardar os segredos como deve ser

O `ClientSecret` é uma password da tua app. A regra: **nunca no git**.

```
# Em desenvolvimento — user-secrets (fora da pasta do repo):
dotnet user-secrets init
dotnet user-secrets set "Authentication:Google:ClientSecret" "GOCSPX-..."
dotnet user-secrets set "Authentication:AzureAd:ClientSecret" "vY0..."

# Em produção — variáveis de ambiente no servidor (o : vira __):
setx Authentication__Google__ClientSecret "GOCSPX-..." /M
```

A configuração do ASP.NET funde tudo — o código lê `Configuration["Authentication:Google:ClientSecret"]` e não sabe de onde veio.

HONESTIDADE SOBRE A NOSSA CASA

Hoje o AsNet (e clones) têm os secrets no `appsettings.json`, commitados. Funciona, mas é dívida: qualquer pessoa com acesso ao repo consegue fazer-se passar pela app. Quando houver oportunidade, migrar para user-secrets/env vars e **rodar** os secrets expostos (gerar novos na consola, apagar os antigos).

---

5. A implementação recomendada (a via da framework)

Não construas o protocolo à mão — o ASP.NET traz handlers que fazem a dança toda. A tua parte são três blocos:

5.1 — Registar os esquemas (Program.cs)

```
builder.Services
    .AddAuthentication(options =>
    {
        // Quem lê a sessão em cada pedido, e onde os logins externos depositam o resultado:
        options.DefaultAuthenticateScheme = "AppCookie";
        options.DefaultSignInScheme      = "AppCookie";
    })
    .AddCookie("AppCookie", options =>
    {
        options.Cookie.Name = "MinhaApp.Auth";
        options.LoginPath = "/login";                    // para onde mandar anónimos
        options.ExpireTimeSpan = TimeSpan.FromDays(7);
        options.SlidingExpiration = true;
        options.Cookie.SecurePolicy = CookieSecurePolicy.Always;  // só HTTPS
        options.Cookie.SameSite = SameSiteMode.Lax;      // Lax: sobrevive ao redirect do OAuth
    })
    .AddGoogle("Google", options =>
    {
        options.SignInScheme = "AppCookie";              // o resultado vira ESTE cookie
        options.ClientId     = builder.Configuration["Authentication:Google:ClientId"]!;
        options.ClientSecret = builder.Configuration["Authentication:Google:ClientSecret"]!;
        options.CallbackPath = "/signin-google";         // o handler É dono deste path

        options.Events.OnCreatingTicket = async context =>
        {
            // ── O OAuth acabou; isto já é a TUA app. ──
            var email = context.Principal?.FindFirst(ClaimTypes.Email)?.Value;
            if (string.IsNullOrWhiteSpace(email))
            { context.Fail("Não foi possível obter o email."); return; }

            // Autorização é problema TEU: é AQUI que decides se a identidade é bem-vinda.
            var gate = context.HttpContext.RequestServices.GetRequiredService();
            if (!await gate.EmailPodeEntrarAsync(email))
            { context.Fail("Este email não tem acesso à aplicação."); return; }

            // Claims próprias — tudo o que a app precisa por sessão, sem ir à BD a cada pedido.
            var identity = (ClaimsIdentity)context.Principal!.Identity!;
            identity.AddClaim(new Claim("app:user-type", "Subscriber"));
        };

        options.Events.OnRemoteFailure = context =>
        {
            context.HandleResponse();                    // sem stack trace para o utilizador
            context.Response.Redirect("/login?erro=" +
                Uri.EscapeDataString(context.Failure?.Message ?? "Falha na autenticação."));
            return Task.CompletedTask;
        };
    })
    .AddOpenIdConnect("Microsoft", options =>
    {
        options.SignInScheme = "AppCookie";
        options.Authority = "https://login.microsoftonline.com/organizations/v2.0";
        options.ClientId     = builder.Configuration["Authentication:AzureAd:ClientId"]!;
        options.ClientSecret = builder.Configuration["Authentication:AzureAd:ClientSecret"]!;
        options.CallbackPath = "/signin-microsoft";
```

```
        options.ResponseType = OpenIdConnectResponseType.Code;   // authorization code flow
        options.Scope.Clear();
        options.Scope.Add("openid"); options.Scope.Add("profile"); options.Scope.Add("email");
        // multi-tenant ("organizations"): o issuer varia por tenant — valida à mão ou desliga
        options.TokenValidationParameters.ValidateIssuer = false;
        options.Events.OnTokenValidated = context => { /* mesmo papel do OnCreatingTicket */ return Task.CompletedTask; };
    });

// E no pipeline — ORDEM OBRIGATÓRIA, antes de MapControllers/MapBlazorHub:
app.UseAuthentication();
app.UseAuthorization();
```

5.2 — Arrancar o login e o logout

```
[Route("auth")]
public class LoginController : ControllerBase
{
    [HttpGet("entrar")]
    public IActionResult Entrar([FromQuery] string provider, [FromQuery] string returnUrl = "/")
    {
        if (!Url.IsLocalUrl(returnUrl)) returnUrl = "/";     // anti open-redirect
        var scheme = provider == "microsoft" ? "Microsoft" : "Google";
        // Challenge = "trata-me do login com este fornecedor e volta para aqui"
        return Challenge(new AuthenticationProperties { RedirectUri = returnUrl }, scheme);
    }

    [HttpGet("sair")]
    public async Task Sair()
    {
        await HttpContext.SignOutAsync("AppCookie");         // mata a NOSSA sessão
        return Redirect("/");                                // a do Google continua viva — e não é problema nosso
    }
}
```

5.3 — Ler o utilizador no Blazor

```
<CascadingAuthenticationState>  @* no router *@

@* numa página: *@
[CascadingParameter] private Task<AuthenticationState> AuthState { get; set; } = default!;
var user = (await AuthState).User;          // ClaimsPrincipal — email, nome, as tuas claims
```

Nas nossas apps existe uma camada própria (AuthenticatedComponentBase + sessão AES) por cima disto; o princípio é o mesmo — as claims do cookie são a fonte.

6. O que a framework está a fazer por ti

Vale a pena saber, porque é o que terias de escrever «à mão» (e é o que a implementação morta no AppMobileLoginController tentou fazer):

PASSO	O QUE O HANDLER FAZ	SE FOSSE À MÃO…
Convite	Constrói o URL de autorização com `client_id`, `redirect_uri`, `scope`, `response_type=code`, `state` e `nonce`	`QueryHelpers.AddQueryString("https://accounts.google.com/o/oauth2/v2/auth", ...)`
Anti-forja	Gera o `state`, guarda-o num *correlation cookie* e confere no regresso — o callback só é aceite se responder a um pedido NOSSO	Gerar valor aleatório, cifrar (DataProtection), guardar, comparar, expirar
Callback	É dono do `CallbackPath` no middleware — qualquer controller no mesmo path nunca corre	Uma action a ler `?code=&state=`
Troca	`POST` servidor↔servidor ao token endpoint: `code + client_secret → tokens`	`HttpClient.PostAsync(tokenEndpoint, form)`
Validação	Valida o `id_token`: assinatura (chaves públicas JWKS do fornecedor), emissor (`iss`), audiência (`aud` = o teu ClientId), validade (`exp`), `nonce`	É aqui que as versões manuais falham — a do AsNet lia o JWT **sem validar a assinatura**
Identidade	Mapeia claims para `ClaimsPrincipal` e dispara os teus eventos	Chamar o userinfo endpoint com o access_token e montar claims

REGRA PRÁTICA

Implementação manual só para *aprender*. Em produção, handlers da framework — menos código, e os erros de validação que tu nem sabias que existiam já estão tratados.

7. Checklist de segurança

- **Redirect URIs de correspondência exata** registados no fornecedor — nunca wildcards.
- **State sempre** (a framework trata) — sem ele, um atacante consegue «colar» a sessão dele na vítima (login CSRF).
- **Secrets fora do git**, rodados se expostos, com lembrete de expiração (Microsoft expira!).
- **Validar o id_token** — assinatura, iss, aud, exp. De borla com os handlers; mortal se esquecido à mão.
- **Cookies**: `Secure` + `HttpOnly` (default) + `SameSite=Lax` (Strict parte o regresso do OAuth; None só com razão).
- **Anti open-redirect**: valida `returnUrl` com `Url.IsLocalUrl` antes de redirecionar.
- **PKCE**: obrigatório em clientes públicos (mobile nativo, SPA — não guardam segredos); os handlers ASP.NET já o usam por default no fluxo code.
- **Autenticação ≠ autorização**: o Google diz quem é; QUEM PODE ENTRAR decides tu, no evento, contra os TEUS dados (o gate de domínios do T-0036 é exatamente isto).
- **Não peças scopes a mais**; não guardes tokens que não vais usar (`SaveTokens = false` se só precisas do login).

8. Erros clássicos e como diagnosticá-los

SINTOMA	CAUSA TÍPICA	CORREÇÃO
`redirect_uri_mismatch` (Google) / `AADSTS50011` (MS)	O redirect URI do pedido não bate certo com o registado — porta, http vs https, barra final, path	Compara caráter a caráter; regista dev E prod
`invalid_client` na troca do code	Secret errado, expirado, ou com espaços/aspas apanhados no copy-paste	Gerar novo secret e substituir
`Correlation failed`	O correlation cookie perdeu-se: SameSite=Strict, site em http, ou o utilizador demorou >15 min na página do Google	SameSite=Lax + HTTPS sempre (mesmo em dev)
Login funciona em dev, morre em produção	Redirect URI de prod não registado; secret de prod diferente; app atrás de proxy sem `ForwardedHeaders` (o handler constrói o redirect com http://)	Registar URI de prod; configurar UseForwardedHeaders
Email vem vazio (Microsoft)	Contas sem `mail` preenchido	Fallback: `preferred_username` / `upn` (o AsNet já faz)
Login «morreu sozinho» num dia qualquer	Client secret da Microsoft expirou	Novo secret; pôr lembrete no calendário desta vez

9. Mapa para o AsNet

CONCEITO DESTE GUIA	NO ASNET2.0
Registo dos esquemas (5.1)	`Program.cs:555-663` — cookie `AsNet.AppMobile` + AddGoogle + AddOpenIdConnect
Arrancar o login (5.2)	`AppMobileLoginController.cs:54` — `/auth/mobile-login` → Challenge
O teu código no evento (5.1)	`OnCreatingTicket` / `OnTokenValidated` — resolver empresa, provisionar Subscriber, claims
O gate de autorização (7)	`AppMobileCompanyResolver.ResolveForExternalEmailAsync` — hoje lê config; T-0036 muda a fonte para `EntitySiteDomain`
A versão manual (6)	A metade morta do controller — builders/callbacks/SaveSubscriberAsync sem chamadores
Login sem OAuth	`/auth/mobile-internal-login` — BCrypt contra a nossa BD, mesmo cookie

10. Glossário

id_token	JWT assinado que descreve a pessoa (OIDC). Serve para LOGIN.
access_token	Chave para chamar APIs do fornecedor em nome da pessoa (ex.: Graph). Não é prova de login.
refresh_token	Troca-se por novos access_tokens sem incomodar a pessoa. Para login puro, dispensável.
scope	O que a app pede para saber/fazer. `openid email profile` = identidade básica.
claim	Um facto sobre a pessoa (email, nome, ou os teus próprios: empresa, tipo de utilizador).
state / nonce	Valores anti-forja: state amarra o callback ao pedido; nonce amarra o id_token ao pedido.
Challenge	«Framework, trata do login com este esquema.» O ponto de partida de tudo.
SignInScheme	Onde o resultado do login externo é depositado — tipicamente o teu cookie.

Guia escrito a 2026-09-04, verificado contra o código real do AsNet2.0 (master) e a configuração de produção. Par: OAuth_Flow_AsNet.pdf.